CODE SPORT

Sandya Mannarswamy

This month's column continues the discussion on data storage systems, with the focus on the metadata structures of file systems.

Last month we discussed how errors can occur in file systems and storage stacks. We also covered 'fsck' – the file system consistency checking tool. In this month's column, let's discuss how file system consistency is maintained in case of 'update in place' file systems.

The most critical piece of information associated with file systems is file system metadata. This represents information about a file such as where its data blocks are located, the access rights, the creator of the file, the time at which the file was modified/accessed, etc. The metadata also includes information like the root of the file system, its block size and mount point, etc. The major data structures used to represent file system metadata information include superblock, inode and dentry.

To recall last month's discussion on the 'Virtual File System (VFS)' in Linux, VFS provides a generic file system abstraction layer over any low level file system layer. The generic file system operations like *read, write, create*, etc are mapped by VFS onto underlying operations of the specific file system. The metadata structures at the VFS layer include superblock, inode and dentry. Each of these data structures supports a set of operations. For instance, superblock contains a set of superblock operations known as 'super_operations'. These operations are defined by the specific file system layer corresponding to that super-block. And they represent the functions/methods which the Linux kernel invokes for each of these specific objects, based on the underlying file system. This allows the kernel VFS layer to be transparent to the underlying file system internals while supporting generic interfaces.

When we discuss file system internal data structures, one point to note is that since file system metadata is maintained on disk as part of the file system itself, each of these metadata structures has both a disk representation and in-memory representation. The in-memory representation is constructed from the on-disk representation of the data structure.

The superblock data structure contains information about a specific mounted file system. There is one superblock object for each file system mounted. There is an in-memory representation of the superblock as well as on-disk representation. The on-disk superblock is typically stored on a specific position on the disk associated with that file system, which enables the kernel to access the disk superblock when the file system is mounted and to construct the in-memory representation of it. In the Linux kernel, the superblock object is represented by 'struct superblock' and is defined in *<linux/fs.h>*.

A superblock object for a file system is created when the file system is mounted by the kernel. For the Linux kernel, this is done by means of the *'alloc_super'* function which is defined in *fs/super.c*. 'Struct superblock' contains a structure field member called *'s_op'*. This is a pointer to a structure of type *'structure super_operations'*, which defines various functions operating on the file system such as *'put_super'*, *'write_super'*, *'sync_fs'* and various inode related operations such as *'alloc_inode'*, *'destroy_inode'*, *'dirty_inode'*, etc.